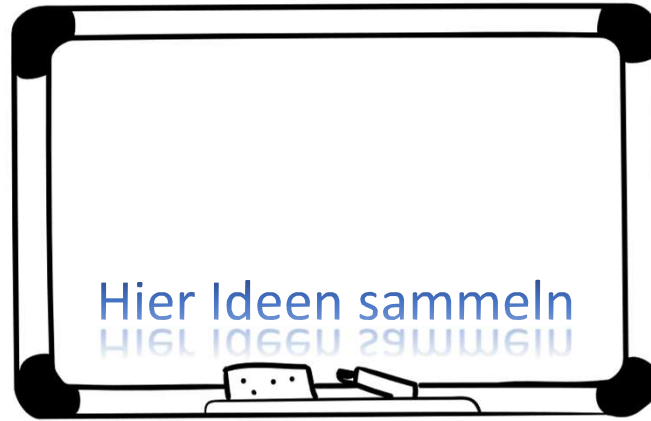


Einführung

Was ist Architektur?

Brainstorming - Was ist Architektur?



Brainstorming mit Teilnehmer, z. B. am Whiteboard,
<https://miro.com/de/>, <https://excalidraw.com>
(und Ergebnis hier als Folie einfügen)

- Strukturierung vs. Code?
- Best Practices, Clean Code?
- Teilbereich von SW-Qualität

Mögliche Antworten

"eine strukturierte oder hierarchische Anordnung der Systemkomponenten sowie Beschreibung ihrer Beziehungen"

(Helmut Balzert)

"Strukturen eines Softwaresystems..."

(Paul Clements)

- Wenn wir nach möglichen Antworten googeln bekommen wir viele Vorschläge.
- Wer ist Helmut Balzert?
 - Schrieb 2 dicke blaue Schinken, quasi das Kompendium der SW-Entwicklung
- In ihrem Kontext mögen alle Antworten korrekt sein
- Häufig **fehlt jedoch der Projektbezug**

<https://youtu.be/CG6itx96wq8?t=172>

Ein paar mögliche Antworten

Wie entwickeln wir Software?



- Quellcode ist die Essenz (Root-Cause)
- Business Logik codieren
- Wir könnten alles in eine Datei schreiben
- Mögliches Problem: **Hohe Komplexität**
-> Modularisierung des Systems



- Wäre möglich, z. B. VS Code, Windows, alles in eine Datei
- Ist es eine gute Idee?
- Wie werden Probleme unterteilt, also modularisiert?

Modularisierung des Systems

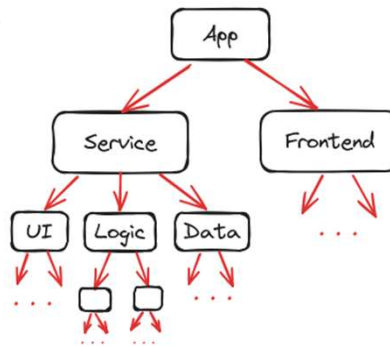


Wie **reduzieren** wir Komplexität?

- System rekursiv in "kleinere Probleme" zerteilen
- Bis **sinnvolle** Granularität erreicht wurde

Tipps, um das zu erreichen:

- **Einheitliche** Modularisierung
- Hohe (Code-)Kohäsion
- Geringe Kopplungen
- Abhängigkeiten reduzieren



- Definition des Systems nach EVA-Prinzip: Eingabe, Verarbeitung, Ausgabe

- Eine Modularisierung ist immer notwendig, egal ob Micro-Services, Client-Server, Monolith etc.

- Schaubild demonstriert Strukturierung

- Wir haben Schichten, Assemblies, Komponenten, Klassen, Methoden usw.

- Das ganze bauen wir mal von unten auf und überlegen uns welche Ebenen haben welche Zuständigkeiten

Anmerkung

<https://www.youtube.com/watch?v=9bBydhSBI3w>

Welche Art der Modularisierung besser ist kann man nicht sagen.

- Bsp: Erde aufteilen je Kontinent vs. Erde aufteilen in Land und Wasser

- Anhaltspunkt SW: Aufgabenbasierte Systeme

- Daran können wir uns entlang hangeln

- Darauf kommt es an:

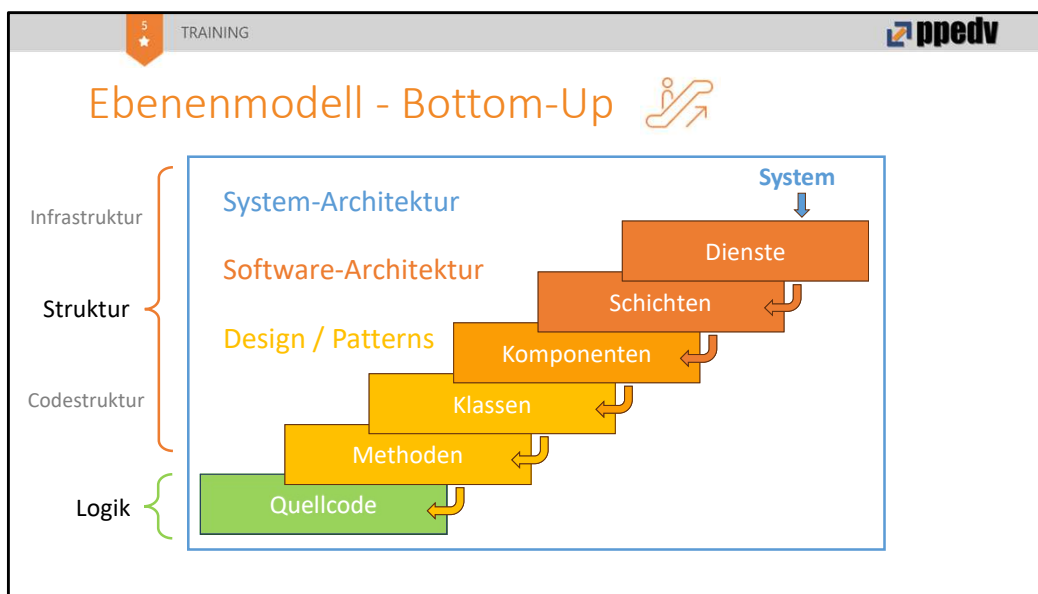
- Einheitliche Modularisierung

- Strategie die für unser Projekt bestmöglich zugeschnitten ist

- max. hohe Kohäsion (Code) und möglichst geringe Koppelung

- vgl. SOLID

Struktur, Qualität und Rollen



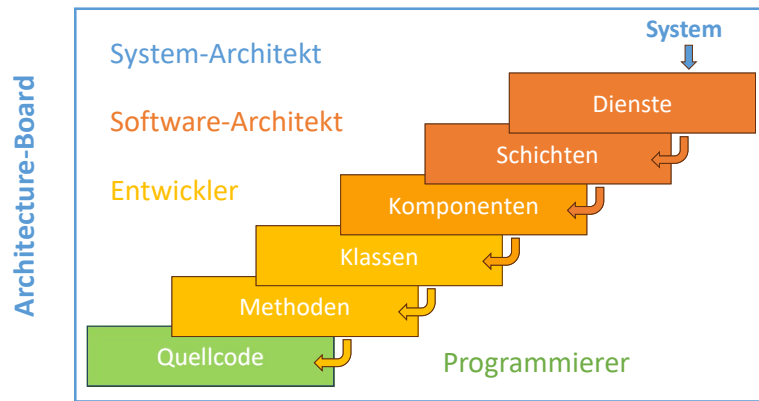
1. Alles dreht sich um die Codierung von Logik.
2. Quellcode wiederverwendbar machen, deshalb Methoden.
(weil alles in eine Datei schreiben doof ist)
3. Methoden thematisch in Klassen strukturieren [oop]
(weil sehr viele Methoden und LoC)
4. Klassen als Komponenten [dll]
(Anzahl Klassen kann auch ausufern)
5. Dann Schichten / Tiers
6. Dienste, Load Balancer etc.
7. System ist die Summe der Bestandteile
8. Der Quellcode kümmert sich nur um die **Logik**.
9. Der Großteil ist Struktur und eigentlich optional
Bsp.: Theoretisch könnten wir das System in eine einzige Datei schreiben.

Aber wo genau ist hier die Architektur?

10. **Logik** & Programmierung wie Bedingungen und Schleifen
- Der unterste Teil, mit dem wir die Business Needs umsetzen
11. **Design**: Nicht funktionale Anforderungen wie...
- Wiederverwendbar, Austauschbar, Testbar, Analysierbarkeit;
- Hier können wir "Erfahrungsrezepte", sog. Entwurfsmuster verwenden
- Bsp. Singleton, Observer, Factory usw.
- Nicht Architektur, aber gehört dazu
12. **SW-Architektur**: Oberhalb des Designs
- Gibt Strukturierung des Dienstes vor
- Wie sind Schichten aufgeteilt

- Wie kommunizieren die Komponenten
 - Laufzeitverhalten der Komponenten usw. usf.
 - Bsp. für Architekturen: Monolithisch (Spaghetti), N-Layer, Komponentenarchitektur
13. System-Architektur: Orchestrierung der Dienste
- Hier auch Patterns wie Monolith, Client-Server usw.

Ebenenmodell - Rollen im Team



- Programmierer sind nur in der Logikschicht zu Hause (Umschüler)
- Entwickler verstehen zusätzlich die Design-Schicht (Studierte)
 - aber sollten auch ein Grundverständnis von Architektur mitbringen, weil Design auch zur Architektur gehört
- SW-Architekt
- System-Architekt
- Mehrere Architekten bilden das Architecture-Board (z. B. ein System-Architekt mit je 2 SW-Architects und Entwickler-Teams)

Was ist jetzt Architektur?

- Hierarchische Strukturierung eines SW-Systems
- Abzielung auf **qualitative Aspekte** für langfristige Projektziele
- Umfasst alle strukturelevanten Ebenen
- Partizipierende Rollen sind
System- und Softwarearchitekten sowie Entwickler

Softwarequalität

qualitative, nicht-funktionale Aspekte:

- Verfügbarkeit,
- Zuverlässigkeit,
- Wartbarkeit,
- Skalierbarkeit

Was sind qualitative Aspekte und warum ist Architektur wichtig?

Warum Softwarequalität?

- Stellenwert der **Architektur**
- Weitere Bereiche der SWQ
- SW-Qualitätsmodell (ISO 9126)
- Tragende Aspekte der **Architektur**



https://de.wikipedia.org/wiki/ISO/IEC_9126

Agenda

- Architektur ist Unterbereich der Qualität
- Die Norm ISO/IEC 9126 stellt eines von mehreren Modellen dar, um Softwarequalität sicherzustellen.
- Final beantworten, warum Architektur so wichtig ist

SW-Qualität – 1. Aspekte identifizieren



- Wartbarkeit
- Skalierbarkeit
- Verfügbarkeit
- Performance
- Testbarkeit
- Lesbarkeit
- Analysierbarkeit
- ...

1. Aufgabe:

Relevante Aspekte identifizieren (für Projekt)

Alleine hilft das aber nicht, deshalb Schritt 2

Bsp. Analysierbarkeit: dann Richtlinie, dass Methode max. zyklomatische Komplexität von 7 haben soll

- Durch Verbindlichkeit kann ich sagen: Bitte Entwickler korrigiere die Methode mit ZK von 15 auf 7

SW-Qualität – 2. Richtlinien

- Auf Richtlinien festlegen
- Metriken nutzen
 - Zyklomatische Komplexität (10-15)
 - Code Coverage (70%)
 - Akzeptanztests



- RL sind Basis der SW-Entw. für gute Grundlage
- Ohne RL für Entw. und Archt. gilt Anarchie!

Zyklomatische Komplexität nach McKab (empf. 15)

SW-Qualität – 3. Analysieren



- Richtlinien sollen gelebt werden
- Automatisiert mit Tools
 - Linter
 - JArchitct
 - NDepend
 - SonarQube
- Manuell mit Code Reviews

- RL auf Einhaltung prüfen
- Man sagt das es "gelebt" wird
- Entweder manuell oder teils automatisiert mit Tools

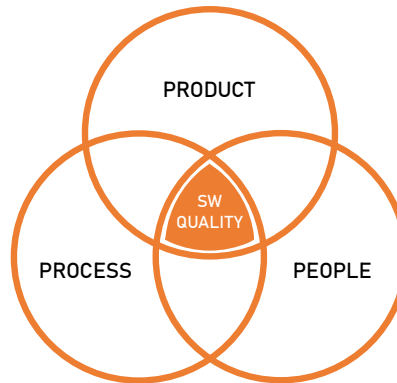
SW-Qualität – Vorgehensmodelle und Prozesse

- Traditionelles Wasserfallmodell
 - Ursprünglich aus Fertigungstechnik übernommen
 - Sequentieller **Pull-Prozess**: Jede Phase muss abgeschlossen sein
 - Einmal konzipiert, immer gleicher Outcome
 - **Probleme bei SW: Unflexibel und spätes Feedback vom Kunden**
- Agile Methoden in der SW-Entwicklung
 - **Push-Prozess**: Kontinuierliche Lieferung von Funktionalitäten
 - Offen und flexibel für neue oder sich ändernden Anforderungen
 - Beispiele: Scrum, Kanban, Scrumban, OKRs, Lean, XP, FDD, DSDM

- Wasserfallmodell war in 90ern sehr populär.
- Historischer Kontext: Industrielle Fertigung.
- Bsp. Kunde der Schaltschränke fertigt:
 - Kann das Produkt am Markt bestehen?
 - Ja? >> Konzept, Architektur z. B. CAD-Zeichnung etc., dann Fertigung, Test, Auslieferung
- Weil Wasserfall funktionierte wurde es für SW-Entwicklung übernommen.
- Hat nicht so gut funktioniert, deshalb daraus gelernt und heute **agile Methoden**
- Unterschied: Fertigung == **Pull Prozess** vs. SW-Dev == **Push Prozess**
- Push Prozess erfordert hohe Flexibilität, nicht nur an Produktqualität
- Agile Methoden
 - <https://mooncamp.com/de/blog/agile-methoden>
 - <https://www.appvizer.de/magazin/organisation-planung/projektmanagement/agile-methoden>

SW-Qualität >> 3P

1. Produktqualität
Eigenschaften der SW von **funktionalen** und **nicht-funktionalen** Merkmalen.
(Architektur und Struktur)
2. Prozessqualität
Fokus auf Methoden und Verfahren;
trägt massiv zur Qualität bei.
(Sprints, CI/CD, Reviews)
3. People (Team)
Fähigkeit, Erfahrungen und reibungsarme,
eingespielte Zusammenarbeit.



Die Kombination dieser drei Bereiche ist entscheidend für die Gesamtqualität der Software. Ein ausgewogener Ansatz, der Produkt, Prozess und People berücksichtigt, führt zu hochwertigen, zuverlässigen und benutzerfreundlichen Softwarelösungen.

Produkt (Beispiele)

- **Funktionalität:** Korrektheit, Vollständigkeit
 - **Zuverlässigkeit:** Fehlertoleranz, Wiederherstellbarkeit
 - **Effizienz:** Ressourcennutzung, Zeitverhalten
 - **Wartbarkeit:** Modularität, Analysierbarkeit
 - **Benutzerfreundlichkeit:** Bedienbarkeit, Erlernbarkeit
- > Hier steckt die Architektur und Struktur drin

Prozess (Schlüsselemente)

- Definierte Phasen und Meilensteine
- Kurze Entwicklungszyklen, um auf Kunden-Feedback reagieren zu können (Iterationen / Sprints)
- CI/CD
- Code Reviews

People (Wichtige Aspekte)

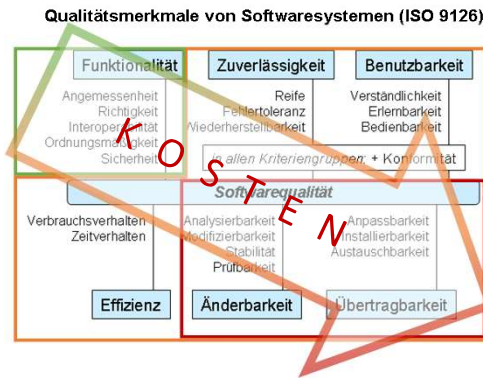
- Eingespielt und reibungsarm
- Effektive Kommunikation und Zusammenarbeit
- Techn. Kompetenz und Fachwissen
- Kontinuierliche Weiterbildung
- Motivation und Engagement
- Diversität und komplementäre Fähigkeiten

Softwarequalitätsmodelle

- Beispiel [ISO/IEC 9126](#)

- **Funktional** (20%)
 - gut kalkulierbar

- **Nicht-Funktional** (80%)
 - Kunden
 - Entwickler



<https://youtu.be/nBbCV7B8aIQ?t=615>

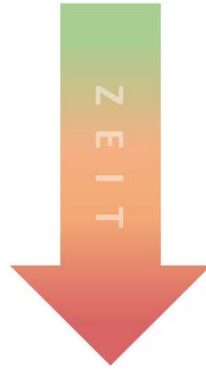
- Wie bei Patterns (z. B. Singleton) gibt es vordefinierte Modelle, die wir verwenden können
- Bei Funktionale Anforderungen meldet sich Kunde sofort bei Fehlern und Nicht-Erfüllung
 - Korrektur vergleichsweise günstig
 - Einfach weil idR punktuelle Anforderungen
- Beispiel Effizienz: Fällt anfangs bei kleiner DB nicht auf
- Rote Aspekte: Wann fallen sie auf?
 - Schwierig...
 - Sichtbar durch Seiteneffekte z. B. Implementierungsaufwand, Bugs...
 - Kann so teuer werden: Neuentwicklung statt Refactoring notwendig!

Warum ist Architektur so wichtig?

- Funktional
(Quellcode)

- Nicht-Funktional

- Zuverlässigkeit
- Benutzbarkeit
- Effizienz
- Änderbarkeit
- Übertragbarkeit



Risiken

- Zeit ↑
- Kosten ↑
- Aufwand ↑
- Seiteneffekte

Architektur

Weniger schlecht programmieren

- Praktische Anleitung zur Verbesserung von Programmierfähigkeiten
- Fokus auf häufige Fehler und Vermeidungsstrategien in der Softwareentwicklung
- Unkonventioneller und humorvoller Ansatz zur Vermittlung von Programmier-Best-Practices
- Sprachunabhängige Betrachtung von Entwicklungskonzepten und Softwarequality



<https://www.oreilly.com/library/view/weniger-schlecht-programmieren/9783955615697>